Practice Set : Operators in C++

Section 1: Conceptual & Practical Questions (1–10)

Question 1: Define an **Operator** and **Operands** in C++. Outline the classification hierarchy of C++ operators based on operand count (Unary, Binary, Ternary) with code syntax examples for each category.

Question 2: Explain the core **Unary Operators** in C++ (++, --, unary -, !, &, *, sizeof). Detail what memory metadata or value modification each operator yields.

Question 3: Explain the execution mechanics and difference between **Prefix Increment** (++x) and **Postfix Increment** (x++). Why does writing chained modifications like a++ + ++a cause undefined behavior in C++?

Question 4: List the five Arithmetic Binary Operators. Explain how integer division performs truncation and why the Modulus operator (%) requires integral operands.

Question 5: Explain the concept of **Short-Circuit Evaluation** for Logical AND (&&) and Logical OR (||) operators. Provide a scenario where evaluating the first operand prevents the second operand from executing.

Question 6: Detail the six **Bitwise Operators** (&, |, ^, ~, <<, >>). Provide a truth table for Bitwise AND, OR, XOR, and NOT across all 2-bit combinations (0 and 1).

Question 7: Step through the binary shift mechanics of right-shifting decimal 47 by 2 bits (47 >> 2) using an 8-bit binary representation. Explain the architectural padding rule applied when right-shifting positive signed numbers.

Question 8: Explain the **Conditional / Ternary Operator** (?:) syntax, logic flow, and operand requirements. What is the best practice regarding nested ternary expressions?

Question 9: Differentiate between the Bitwise AND operator (&) and the Address-of unary operator (&), as well as Bitwise OR (|) vs. Logical OR (||) in terms of operands and functionality.

Question 10: Write a complete C++ program that demonstrates prefix/postfix increment, bitwise right-shift, short-circuit logical evaluation, and ternary operator value selection.

Section 2: Interview & Viva Questions (11–20)

Question 11 (Viva): How does sizeof() evaluate its result in memory, and does it execute expressions inside it (e.g., sizeof(a++)) at runtime?

Question 12 (Viva): What happens during Short-Circuit evaluation of (x != 0) && (10 / x > 1) if x equals 0?

Question 13 (Viva): Why is postfix increment (x++) generally considered slightly less efficient than prefix increment (++x) when used on non-primitive objects or iterators?

Question 14 (Viva): What is the exact decimal result of right-shifting decimal 47 by 2 bits (47 >> 2)?

Question 15 (Viva): Which bitwise operator can be used to toggle/invert specific bits of an integer without altering other bits?

Question 16 (Viva): What error occurs when applying the modulus operator (%) to float or double operands in C++?

Question 17 (Viva): What does the unary address-of operator (&) return when applied to a variable in RAM?

Question 18 (Viva): What is the only operator in C++ that accepts exactly three operands?

Question 19 (Viva): Why is writing int result = a++ + ++a; considered unsafe coding practice in C++?

Question 20 (Viva): What padding rule applies to leading empty bit slots when right-shifting positive signed integers in C++?

Answers & Explanations

Answer: 01

Operator: A dedicated compiler symbol instructing the Central Processing Unit (CPU) or Arithmetic Logic Unit (ALU) to execute a specific mathematical, logical, relational, or bitwise computation.

Operand: Data items operated upon, which can be literal constants, variables, or expressions.

Classification Hierarchy:

Unary Operators (1 Operand): e.g., ++a, -x, !flag.

Binary Operators (2 Operands): e.g., a + b, x && y.

Ternary Operator (3 Operands): e.g., (a > b) ? a : b.

Answer: 02

++ (Increment): Increases value by 1.

-- (Decrement): Decreases value by 1.

- (Unary Minus): Negates numerical sign.

! (Logical NOT): Inverts boolean truth value.

& (Address-of): Returns physical RAM address pointer of variable.

* (Indirection/Dereference): Accesses value stored at target address.

sizeof: Returns exact compile-time size of type/variable in bytes.

Answer: 03

Prefix (++x): Increments FIRST, returns NEW value immediately.

Postfix (x++): Copies OLD value, increments SECOND, returns OLD copy.

Undefined Behavior: Chaining multiple increment/decrement modifications on the same variable within a single expression (e.g., a++ + ++a) causes unsequenced side effects, leading to undefined execution behavior in C++.

Answer: 04

Arithmetic Operators: Addition (+), Subtraction (-), Multiplication (*), Division (/), Modulus (%).

Integer Division: Truncates fractional/decimal components (e.g., 20 / 4 is 5, 10 / 3 is 3).

Modulus (%): Calculates the remainder of integer division (e.g., 10 % 3 is 1). It requires integer operands and fails to compile on floating-point types.

Answer: 05

Short-Circuit Execution:

Logical AND (A && B): If A is false, B is **NEVER** evaluated because the overall result is guaranteed to be false.

Logical OR (A || B): If A is true, B is **NEVER** evaluated because the overall result is guaranteed to be true.

Scenario: In (x != 0) && (10 / x > 1), if x == 0, the first operand is false, so (10 / x > 1) is bypassed, preventing a division-by-zero runtime crash.

Answer: 06

Bitwise Operators: Bitwise AND (&), Bitwise OR (|), Bitwise XOR (^), Bitwise NOT (~), Left Shift (<<), Right Shift (>>).

Truth Table:

Bit A	Bit B	A & B (AND)	A | B (OR)	A ^ B (XOR)	~A (NOT)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Answer: 07

Binary Conversion: Decimal 47 in 8-bit binary is 00101111.

Right Shift by 2 (47 >> 2):

Original: [0][0][1][0][1][1][1][1]

Dropped rightmost bits: 11

Shifted Result: [0][0][0][0][1][0][1][1]

Decimal Value: $8 + 2 + 1 = 11$.

Padding Rule: When right-shifting positive signed integers, empty slots on the left are padded with leading zeros.

Answer: 08

Syntax: (Condition) ? expression_if_true : expression_if_false;

Logic Flow: Evaluates the condition; if true, executes and returns Operand 2; if false, executes and returns Operand 3.

Best Practice: Keep ternary expressions concise for simple inline assignments. Avoid nesting multiple ternary operators; use if-else blocks for complex branching to maintain code readability.

Answer: 09

Unary & vs Binary &: Unary & requires 1 operand and returns physical RAM memory address. Binary & requires 2 operands and performs bit-level logical AND.

Bitwise | vs Logical ||: Bitwise | operates on individual bits of integer operands without short-circuiting. Logical || operates on boolean expressions and uses short-circuit evaluation.

Answer: 10

```
#include <iostream>
int main() {
   int val = 10;

   // 1. Prefix and Postfix Increment
   int preResult = ++val;   // val becomes 11, preResult = 11
   int postResult = val++;  // postResult = 11, val becomes 12

   // 2. Bitwise Right-Shift
   int x = 47;
   int shifted = x >> 2;    // 47 >> 2 = 11

   // 3. Short-Circuit Evaluation
   int divisor = 0;
   bool safeCheck = (divisor != 0) && (100 / divisor > 2);

   // 4. Ternary Operator
   int a = 15, b = 20;
   int maxVal = (a > b) ? a : b;

   std::cout << "Val: " << val << " | Pre: " << preResult << " | Post: " << postResult <<
"\n";
   std::cout << "Bitwise Shift (47 >> 2): " << shifted << "\n";
   std::cout << "Safe Check Result: " << std::boolalpha << safeCheck << "\n";
   std::cout << "Max Value: " << maxVal << "\n";

   return 0;
}
```

Answer: 11

sizeof() is a compile-time operator that inspects data types to return byte allocations. It does **not** evaluate expressions at runtime; thus, sizeof(a++) returns the byte size of a without incrementing a.

Answer: 12
Short-circuit logic detects that (x != 0) is false. It immediately evaluates the entire expression to false without executing (10 / x > 1), avoiding a division-by-zero error.

Answer: 13
Postfix increment creates a temporary copy of the object's previous state before incrementing and returning that copy, incurring performance overhead compared to prefix increment.

Answer: 14
Decimal 11.

Answer: 15
The Bitwise XOR operator (^).

Answer: 16
A compilation error, because the modulus operator (%) only supports integer operands.

Answer: 17
It returns the physical RAM address pointer of the variable (e.g., 0x7ffd9000).

Answer: 18
The Conditional / Ternary Operator (?:).

Answer: 19
It causes undefined behavior because modifying a single variable multiple times within an unsequenced expression violates evaluation rules.

Answer: 20
Empty leading bit positions on the left are padded with zeros (0).